Facultade de Informática
UNIVERSIDADE DA CORUÑA

TRABALLO FIN DE GRAO
GRAO EN ENXEÑARÍA INFORMÁTICA
MENCIÓN EN INGENIERÍA DEL SOFTWARE

Diseño, despliegue y operación de una plataforma cloud para la gestión de transportes en AWS mediante infraestructura como código y prácticas DevOps

Estudante: Pablo Manzanares López

Dirección: Paula María Castro Castro

A Coruña, abril de 2026.

Agradecimientos

Pendiente de redactar al cierre del trabajo.

Resumen

Este Trabajo Fin de Grado aborda el diseño, despliegue y operación de una plataforma cloud para la gestión de transportes sobre AWS. El sistema se construye como un backend contenerizado con persistencia en PostgreSQL, infraestructura definida mediante Terraform, automatización CI/CD y mecanismos de observabilidad y seguridad básica.

Pendiente de completar con el resumen final cuando el alcance técnico y las evidencias de despliegue estén cerrados.

Abstract

This Bachelor's Thesis addresses the design, deployment, and operation of a cloud-based transport management platform on AWS. The system is built as a containerized backend with PostgreSQL persistence, Terraform-based infrastructure, CI/CD automation, and basic observability and security mechanisms.

To be completed with the final abstract once the technical scope and deployment evidence are closed.

Palabras clave:

- Computación en la nube
- DevOps
- Infraestructura como código
- AWS
- Terraform
- ASP.NET Core
- Gestión de transportes

Keywords:

- Cloud computing
- DevOps
- Infrastructure as code
- AWS
- Terraform
- ASP.NET Core
- Transport management

Índice general

1 Introducción **1**

1.1 Motivación 1

1.2 Problema a resolver 1

1.3 Alcance del trabajo 1

1.4 Estructura de la memoria 2

2 Contexto y objetivos **3**

2.1 Contexto tecnológico 3

2.2 Objetivo general 3

2.3 Objetivos específicos 3

2.4 Criterios de éxito 4

3 Metodología y planificación **5**

3.1 Enfoque de trabajo 5

3.2 Fases del proyecto 5

3.3 Gestión del alcance 5

3.4 Herramientas utilizadas 6

4 Análisis y requisitos **7**

4.1 Actores 7

4.2 Requisitos funcionales 7

4.3 Requisitos no funcionales 7

4.4 Modelo de dominio 7

4.5 Casos de uso principales 7

5 Diseño de la arquitectura **8**

5.1 Visión general 8

5.2 Arquitectura lógica del backend 8

5.3 Arquitectura de datos 8
5.4 Arquitectura cloud en AWS 8
5.5 Decisiones arquitectónicas 8

6 Implementación del backend 10
6.1 Estructura del proyecto 10
6.2 API REST 10
6.3 Persistencia 10
6.4 Autenticación y autorización 10
6.5 Pruebas del backend 10

7 Infraestructura y despliegue 11
7.1 Contenerización 11
7.2 Infraestructura como código 11
7.3 Entorno AWS 11
7.4 Despliegue de la aplicación 11
7.5 Gestión de configuración y secretos 11

8 Integración y despliegue continuo 12
8.1 Objetivos del pipeline 12
8.2 Validación automática 12
8.3 Publicación de artefactos 12
8.4 Despliegue automatizado 12

9 Observabilidad y seguridad 13
9.1 Comprobaciones de salud 13
9.2 Logs y métricas 13
9.3 Alarmas y operación 13
9.4 Seguridad aplicada 13
9.5 Limitaciones de seguridad 13

10 Validación y resultados 14
10.1 Estrategia de validación 14
10.2 Pruebas funcionales 14
10.3 Validación de infraestructura 14
10.4 Resultados obtenidos 14
10.5 Análisis de limitaciones 14

11 Conclusiones **15**
11.1 Conclusiones del trabajo 15
11.2 Competencias aplicadas 15
11.3 Lecciones aprendidas 15
11.4 Líneas futuras 15

A Material complementario **17**
A.1 Anteproyecto 17
A.2 Evidencias técnicas 17

Lista de acrónimos **18**

Glosario **19**

Bibliografía **20**

Índice de figuras

Índice de cuadros

Capítulo 1

Introducción

Este trabajo presenta el diseño, desarrollo, despliegue y operación de una plataforma cloud para la gestión de transportes. El proyecto se plantea como un trabajo clásico de ingeniería en el que el alcance funcional se mantiene deliberadamente acotado para poder profundizar en decisiones de arquitectura, automatización, infraestructura como código, observabilidad y operación en Amazon Web Services (AWS).

1.1 Motivación

Pendiente de redactar. Esta sección deberá explicar por qué una plataforma de gestión de transportes es un caso adecuado para demostrar competencias de ingeniería del software, cloud y DevOps.

1.2 Problema a resolver

Pendiente de redactar. Se describirá la necesidad de gestionar transportes, vehículos, conductores, estados operativos y eventos logísticos desde un backend desplegable y operable en un entorno cloud.

1.3 Alcance del trabajo

El trabajo se centra en un backend API, persistencia relacional, ejecución local reproducible, infraestructura AWS definida con Terraform, pipeline CI/CD y mecanismos básicos de observabilidad y seguridad. Quedan fuera del alcance principal el desarrollo de una interfaz de usuario completa y las funcionalidades avanzadas de planificación logística.

1.4 Estructura de la memoria

Pendiente de completar al final de la redacción. Esta sección resumirá brevemente el contenido de cada capítulo.

Capítulo 2

Contexto y objetivos

2.1 Contexto tecnológico

Pendiente de redactar. Se introducirá el papel de los sistemas backend, los despliegues contenerizados, la infraestructura como código y las prácticas DevOps en sistemas software actuales.

2.2 Objetivo general

Diseñar, implementar, desplegar y operar una plataforma cloud para la gestión de transportes sobre AWS, utilizando infraestructura como código y prácticas DevOps para garantizar reproducibilidad, automatización, seguridad básica y capacidad de observación.

2.3 Objetivos específicos

- Diseñar una arquitectura cloud para una plataforma de gestión de transportes sobre AWS.
- Implementar un backend con funcionalidades básicas de transportes, vehículos, conductores, usuarios y eventos.
- Definir la infraestructura mediante Terraform para obtener entornos reproducibles.
- Contenerizar la aplicación y desplegarla en servicios gestionados de AWS.
- Automatizar validación, construcción y despliegue mediante CI/CD.
- Incorporar logs, métricas, comprobaciones de salud y trazabilidad operativa.
- Aplicar medidas básicas de seguridad en autenticación, autorización, secretos e infraestructura.

- Evaluar el sistema mediante pruebas funcionales y análisis de ejecución.
- Documentar decisiones, limitaciones y líneas de evolución futura.

2.4 Criterios de éxito

Pendiente de concretar. Esta sección recogerá los criterios verificables que permitan demostrar que el trabajo cumple los objetivos anteriores.

Capítulo 3

Metodología y planificación

3.1 Enfoque de trabajo

El proyecto se desarrolla con un enfoque iterativo e incremental. Cada iteración busca producir una versión funcional o verificable del sistema, evitando introducir complejidad que no aporte valor directo a los objetivos del trabajo.

3.2 Fases del proyecto

1. Análisis y diseño del alcance, requisitos, modelo de datos y arquitectura.
2. Desarrollo del backend y de los casos de uso principales.
3. Contenerización y preparación del entorno local reproducible.
4. Definición de la infraestructura cloud mediante Terraform.
5. Automatización de integración y despliegue continuo.
6. Incorporación de observabilidad y medidas básicas de seguridad.
7. Validación funcional y análisis del comportamiento en ejecución.
8. Redacción de memoria, conclusiones y líneas futuras.

3.3 Gestión del alcance

Pendiente de redactar. Se justificará la decisión de priorizar profundidad operativa y calidad de despliegue frente a amplitud funcional.

3.4 Herramientas utilizadas

Pendiente de completar. Se documentarán las herramientas utilizadas: .NET, PostgreSQL, Docker, Terraform, AWS, GitHub Actions, Postman/Newman y herramientas de documentación.

Capítulo 4

Análisis y requisitos

4.1 Actores

Pendiente de redactar. Se describirán los perfiles principales del sistema, diferenciando al usuario administrador y al usuario operador.

4.2 Requisitos funcionales

Pendiente de completar desde `docs/Requirements.md`. Como base, el sistema cubre gestión de transportes, vehículos, conductores, usuarios, asignaciones, transiciones de estado y eventos logísticos.

4.3 Requisitos no funcionales

Pendiente de redactar. Se incluirán requisitos de reproducibilidad, seguridad, mantenibilidad, observabilidad, despliegue automatizado y operación en cloud.

4.4 Modelo de dominio

Pendiente de completar con entidades, relaciones principales y reglas de negocio relevantes.

4.5 Casos de uso principales

Pendiente de redactar. Se documentarán los flujos de autenticación, administración de usuarios, gestión de recursos, asignación, cambio de estado y registro de eventos.

Capítulo 5

Diseño de la arquitectura

5.1 Visión general

Pendiente de redactar. Se presentará la arquitectura global del sistema, diferenciando backend, base de datos, contenedores, infraestructura cloud, pipeline y servicios de observabilidad.

5.2 Arquitectura lógica del backend

Pendiente de completar. Se explicará la organización del backend ASP.NET Core, la separación entre controladores, servicios de aplicación, dominio e infraestructura de persistencia.

5.3 Arquitectura de datos

Pendiente de completar. Se documentará el uso de PostgreSQL, migraciones de Entity Framework Core, borrado lógico, restricciones e índices relevantes.

5.4 Arquitectura cloud en AWS

Pendiente de completar desde `docs/CloudArchitecture.md`. La arquitectura objetivo incluye Route 53, ACM, Application Load Balancer, ECS Fargate, ECR, RDS PostgreSQL, Secrets Manager, Systems Manager Parameter Store y CloudWatch [1].

5.5 Decisiones arquitectónicas

Pendiente de redactar. Esta sección justificará las decisiones principales y sus alternativas razonables, incluyendo el uso de Terraform como herramienta de infraestructura como

código [2].

Capítulo 6

Implementación del backend

6.1 Estructura del proyecto

Pendiente de redactar. Se describirá la solución .NET, los proyectos principales y las responsabilidades de cada carpeta, tomando ASP.NET Core como base del servicio HTTP [3].

6.2 API REST

Pendiente de completar. Se documentarán los endpoints principales y los criterios seguidos para respuestas, validación y errores.

6.3 Persistencia

Pendiente de redactar. Se explicará el uso de Entity Framework Core, PostgreSQL, migraciones y reglas de integridad.

6.4 Autenticación y autorización

Pendiente de completar. Se documentará el arranque del primer administrador, login, emisión de JWT y control de acceso por roles.

6.5 Pruebas del backend

Pendiente de redactar. Se recogerán las pruebas automatizadas y los criterios de validación funcional del backend.

Capítulo 7

Infraestructura y despliegue

7.1 Contenerización

Pendiente de redactar. Se describirá la imagen Docker de la API y el entorno local reproducible mediante Docker Compose, usando contenedors como unidad de empaquetado y ejecución.

7.2 Infraestructura como código

Pendiente de completar. Se explicará la estructura de Terraform, los módulos, los entornos y la estrategia de estado remoto.

7.3 Entorno AWS

Pendiente de redactar. Se documentará la cuenta utilizada, región, nomenclatura, etiquetas, DNS y separación de recursos.

7.4 Despliegue de la aplicación

Pendiente de completar. Se describirá el flujo desde la construcción de la imagen hasta la ejecución en ECS Fargate detrás de un Application Load Balancer.

7.5 Gestión de configuración y secretos

Pendiente de redactar. Se explicará cómo se gestionan parámetros de configuración, credenciales y secretos sin introducirlos en el repositorio.

Capítulo 8

Integración y despliegue continuo

8.1 Objetivos del pipeline

Pendiente de redactar. Se describirá qué garantías aporta el pipeline antes de desplegar cambios.

8.2 Validación automática

Pendiente de completar. Se documentarán restauración de dependencias, compilación, pruebas, validación de migraciones y comprobación de configuración Docker.

8.3 Publicación de artefactos

Pendiente de redactar. Se explicará la construcción y publicación de imágenes en ECR cuando esta parte esté cerrada.

8.4 Despliegue automatizado

Pendiente de completar. Se describirá la estrategia de despliegue desde GitHub Actions hacia AWS, incluyendo autenticación mediante OIDC si aplica.

Capítulo 9

Observabilidad y seguridad

9.1 Comprobaciones de salud

Pendiente de redactar. Se documentarán los endpoints de salud y su uso en entorno local y cloud.

9.2 Logs y métricas

Pendiente de completar. Se explicará la estrategia de logs de aplicación, logs de contenedor y métricas en CloudWatch.

9.3 Alarmas y operación

Pendiente de redactar. Se recogerán alarmas, criterios de operación y evidencias de comportamiento en ejecución.

9.4 Seguridad aplicada

Pendiente de completar. Se documentarán controles de autenticación, autorización, red, secretos, permisos e infraestructura.

9.5 Limitaciones de seguridad

Pendiente de redactar. Se explicarán los límites razonables del trabajo por tratarse de un TFG y no de un producto productivo completo.

Capítulo 10

Validación y resultados

10.1 Estrategia de validación

Pendiente de redactar. Se describirá cómo se validan requisitos funcionales, infraestructura, despliegue y comportamiento operativo.

10.2 Pruebas funcionales

Pendiente de completar. Se documentarán pruebas automatizadas, pruebas de integración y flujos Postman/Newman.

10.3 Validación de infraestructura

Pendiente de redactar. Se recogerán validaciones de Terraform, planificación de cambios y evidencias de despliegue.

10.4 Resultados obtenidos

Pendiente de completar hacia el final del proyecto con capturas, métricas, endpoints desplegados y resultados de ejecución.

10.5 Análisis de limitaciones

Pendiente de redactar. Se identificarán limitaciones técnicas, funcionales y operativas detectadas durante el proyecto.

Capítulo 11

Conclusiones

11.1 Conclusiones del trabajo

Pendiente de redactar al cierre del proyecto. Se resumirá el grado de cumplimiento de los objetivos y el valor técnico del trabajo realizado.

11.2 Competencias aplicadas

Pendiente de completar. Se relacionará el trabajo con competencias de ingeniería del software, arquitectura, bases de datos, cloud, DevOps, seguridad y operación.

11.3 Lecciones aprendidas

Pendiente de redactar. Se recogerán aprendizajes técnicos y metodológicos relevantes.

11.4 Líneas futuras

Pendiente de completar. Se propondrán mejoras razonables: frontend, mayor cobertura funcional, despliegues multi-entorno, endurecimiento de seguridad, pruebas de carga y evolución de observabilidad.

Apéndices

Apéndice A

Material complementario

A.1 Anteproyecto

El anteproyecto firmado se conserva en el repositorio de la memoria como referencia documental del alcance aprobado.

A.2 Evidencias técnicas

Pendiente de completar con evidencias de ejecución, capturas, salidas relevantes de comandos, diagramas complementarios o enlaces a artefactos de validación.

Lista de acrónimos

API Application Programming Interface. 1, 11

AWS Amazon Web Services. 1

CI/CD Continuous Integration / Continuous Deployment. 1

ECR Elastic Container Registry. 8

ECS Elastic Container Service. 8

JWT JSON Web Token. 10

RDS Relational Database Service. 8

Glosario

backend Componente servidor responsable de exponer la lógica de negocio, gestionar la persistencia y ofrecer una interfaz de comunicación a otros clientes o sistemas.. 1

contenedor Unidad de empaquetado y ejecución que incluye una aplicación y sus dependencias para facilitar despliegues reproducibles.. 11

infraestructura como código Práctica consistente en definir recursos de infraestructura mediante ficheros versionables y automatizables.. 1, 8

Bibliografía

[1] Amazon Web Services, "Amazon elastic container service documentation," 2026, consultado el 25 de abril de 2026. [En línea]. Disponible en: https://docs.aws.amazon.com/ecs/

[2] HashiCorp, "Terraform documentation," 2026, consultado el 25 de abril de 2026. [En línea]. Disponible en: https://developer.hashicorp.com/terraform/docs

[3] Microsoft, "Asp.net core documentation," 2026, consultado el 25 de abril de 2026. [En línea]. Disponible en: https://learn.microsoft.com/aspnet/core/